

Stored Cross-Site Scripting:

Stored XSS: WAF Coverage and a Live Demonstration of a Detection Blind Spot

I tested DVWA's Stored XSS module against the ModSecurity WAF, then deliberately excluded WAF coverage for this attack class to observe, and document, what the SIEM does and does not see when its only upstream detection layer is bypassed.

Overview

In this phase, I tested DVWA's Stored XSS module, a guestbook that saves user input to a database and redisplay it to any future visitor, against the lab's ModSecurity WAF, extending the coverage testing already done for SQL injection and command injection. Stored XSS carries a distinct risk profile from those two: a payload that gets past the WAF here isn't just a one time reflected result, it persists and can execute in every subsequent visitor's browser.

After confirming baseline WAF coverage across four distinct XSS techniques, I ran a second, deliberate exercise: temporarily excluding ModSecurity's XSS rule category to observe exactly what a successful attack looks like, and, more importantly, what the SIEM records when its only detection layer for this traffic is bypassed.

Part 1: Baseline WAF Coverage

I submitted four payloads to DVWA's Stored XSS module (Security Level: Low) through the ModSecurity WAF (port 8081), each using a different injection technique to test whether coverage generalized beyond the most obvious pattern.

Payload	Technique	Result
<code><script>alert('xss')</script></code>	Classic script tag	Blocked (403)
<code></code>	Event handler, no <script>	Blocked (403), score 18
<code>click</code>	javascript: URI	Blocked (403)
<code><ScRiPt>alert(String.fromCharCode(...))</sCRipt></code>	Case + charcode evasion	Blocked (403), score 23

All four were blocked, each producing a Wazuh alert on rule 31333 ("ModSecurity rejected a query", level 7), the same generic WAF block rule seen across every attack type tested in this lab so far.

Finding: A Static, Miscategorized MITRE Tag

I inspected the alert detail for these blocks and found a more specific version of the classification gap first identified during command injection testing. Rule 31333 carries a fixed MITRE ATT&CK mapping, Technique T1083 (File and Directory Discovery), Tactic Discovery, baked into the rule definition itself, applied identically regardless of what attack the underlying request actually was.

This mapping does not describe stored XSS in any way: no file or directory enumeration occurred in any of these requests. Because rule 31333 has no awareness of the attack content it wraps, it inherits one static, pre assigned MITRE tag and applies it to every category of blocked request that lacks its own dedicated rule (as command injection and now XSS both do). An analyst triaging by MITRE tactic would see "Discovery" on a blocked stored XSS attempt and be pointed toward the wrong investigative context entirely, a genuine, evidence backed limitation of relying on this rule's metadata at face value.

Part 2: Deliberately Bypassing WAF Coverage

To observe the practical consequence of this single layer detection design, I deliberately and temporarily excluded ModSecurity's XSS rule category using a rule exclusion, not a rule deletion. CRS tags every rule by attack category, and I added a single line to the designated post CRS exclusion file:

```
SecRuleRemoveByTag "attack-xss"
```

This does not remove, delete, or modify any of the underlying OWASP Core Rule Set files, every XSS detection rule remained loaded and active. The exclusion simply instructs ModSecurity to discard the disruptive action for any rule tagged attack-xss after it has already evaluated the request, functionally equivalent to a real world overly broad exception rule (a common misconfiguration pattern in production WAF deployments, typically introduced to resolve a false positive without adequately scoping the exception).

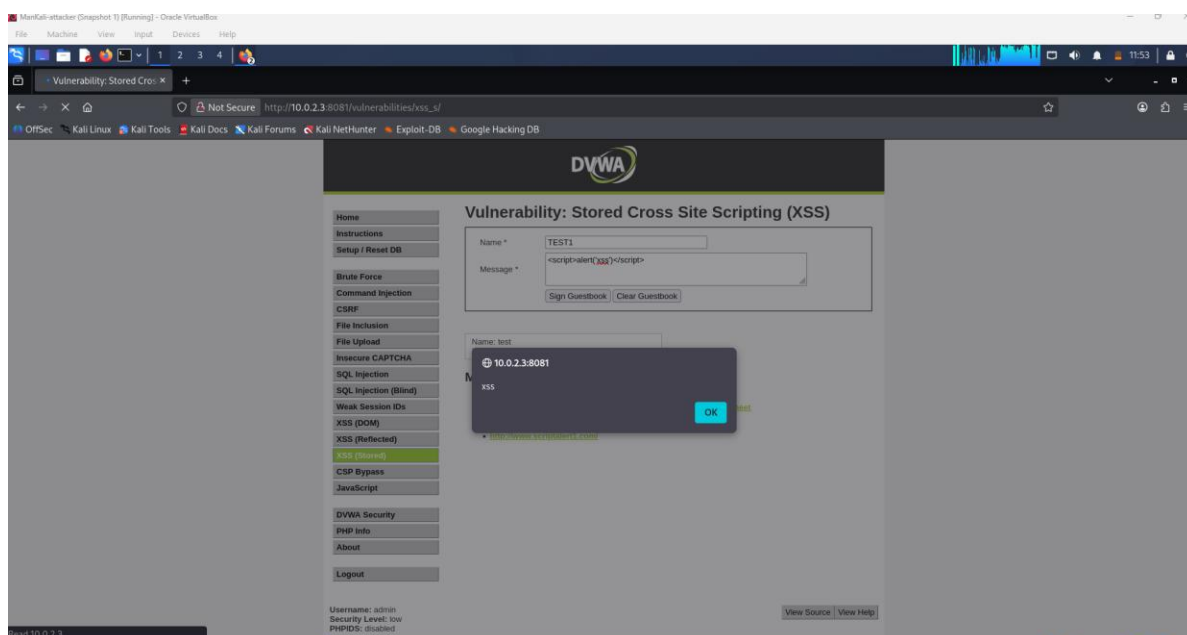
I took a backup of the original (unmodified) exclusion file before this change, and fully reverted the change at the end of the exercise.

Result: A Working Exploit and a Silent SIEM

With the exclusion active, I resubmitted the classic payload:

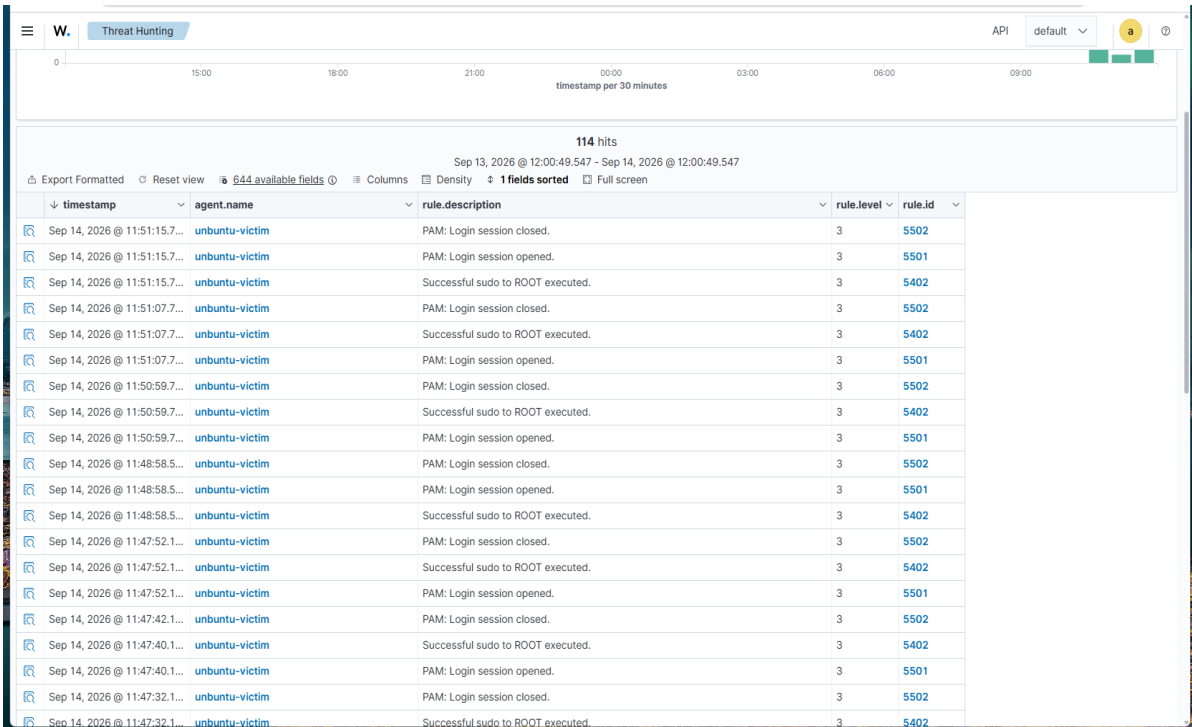
```
<script>alert('xss')</script>
```

The request returned 200 instead of 403, and reloading the guestbook page produced a genuine, executing JavaScript alert box in the browser, confirmed proof of concept execution, not merely a saved database entry.



A live, executing XSS payload: the alert box fires on every subsequent page load once the WAF's XSS category is excluded, confirming genuine stored and executed script injection.

I then checked Wazuh for any trace of this successful attack, filtered to the victim host and sorted by newest across the full attack window.



Wazuh's alert stream at the time of the successful, unblocked XSS attack: no ModSecurity entry, no rule 31333, no reference to the attack at all, only routine, unrelated host activity (PAM logins, sudo events).

My search returned nothing related to the attack whatsoever. No block, no partial detection, no anomaly flag of any kind, complete silence in the SIEM for a confirmed, successfully executed attack.

Finding: A Single Point of Failure in the Detection Pipeline

This is the most significant finding of this phase, and a more consequential one than any individual bypass technique tested elsewhere in this lab. Wazuh's visibility into web application attacks in this architecture is entirely downstream of, and dependent on, ModSecurity choosing to block a request. There is no independent, secondary layer, no application level logging, no browser side telemetry, no behavioral detection, watching this traffic. When the WAF's coverage for a given attack category is disabled or bypassed, the SIEM does not degrade gracefully or flag reduced visibility; it goes completely and silently blind to that entire class of attack.

This distinguishes a bypass finding from an architectural finding: an attacker does not need to defeat detection on every request, or even find a sophisticated evasion technique, a single successful misconfiguration of the WAF's own exclusion rules (a change that, notably, produces no alert or warning of its own) is sufficient to remove detection entirely for that attack category, with no secondary layer to catch what was missed.

I reverted and re verified the exclusion at the end of testing: resubmitting the identical payload returned the expected 403, confirming full protection and detection were restored before concluding the exercise.

MITRE ATT&CK Mapping

Framework	Mapping
MITRE Technique	T1059.007, JavaScript (Command and Scripting Interpreter)
MITRE Tactic	Execution
Related Technique	T1190, Exploit Public Facing Application (initial vector)

Results Summary

- Tested 4 stored XSS payloads across distinct techniques (tag based, event handler, URI based, case/encoding evasion), all blocked by ModSecurity's CRS
- Identified that Wazuh's generic WAF block rule (31333) carries a static, incorrect MITRE tag (T1083 / Discovery) applied identically to any attack type lacking its own dedicated classification rule
- Deliberately excluded WAF XSS coverage via a scoped rule exclusion (not deletion), and confirmed a real, executing XSS payload succeeded as a result
- Confirmed the SIEM produced zero alerts or log entries of any kind for the successful attack, a complete, silent detection gap
- Identified this as a single point of failure architectural finding: detection depends entirely on the WAF layer, with no independent secondary detection for web application attacks
- Fully reverted the exclusion and re confirmed blocking and detection were restored before concluding testing

Skills Demonstrated

- Stored XSS testing methodology across multiple injection and evasion techniques
- Precise technical distinction between a rule exclusion and a rule deletion, and its real world relevance (overly broad exception rules as a common production misconfiguration)
- Safe, reversible security control manipulation in a lab environment: backup, controlled change, verification, full rollback
- Detection architecture analysis: identifying a single point of failure across a multi tool pipeline, not just a single missed signature
- SIEM gap analysis distinguishing 'attack blocked but misclassified' from 'attack succeeded and produced zero telemetry'
- MITRE ATT&CK mapping and critical evaluation of a SIEM's own automated rule metadata rather than trusting it at face value